

Mapa de Sabores - Propuesta de Proyecto Final

Proyecto Final Desarrollo de Sistemas Web

Alumno: Diego Rafael Guaraz

1. Resumen

Mapa de Sabores es un sistema web interactivo de descubrimiento gastronómico basado en una arquitectura híbrida de **Base de Datos Relacional + Inteligencia Artificial (IA)**.

El proyecto resuelve el problema del maridaje e innovación culinaria mediante una representación en forma de **Grafo de Sabores** interactivo y dinámico. Permite a profesionales de la cocina, estudiantes de gastronomía y aficionados explorar combinaciones de ingredientes basados en afinidad química y culinaria, consultar recetas reales integradas y recibir explicaciones organolépticas generadas por modelos de lenguaje (LLM).

Principales Pilares de Ingeniería:

1. **Certeza en el Core (Base Relacional Estable):** La red de sabores reside en una base de datos PostgreSQL optimizada con índices compuestos bidireccionales. La estructura del grafo se define por datos curados y validados, garantizando consistencia, respuestas instantáneas (< 10ms) y cero alucinaciones en la navegación UI.
 2. **Pipeline de Datos Sintéticos Offline:** Un pipeline de ingeniería de prompts sobre LLMs compila y cura un dataset inicial de ~250 ingredientes y ~1,500 pares de afinidad.
 3. **Capa de IA Desacoplada e Intercambiable:** Un servicio backend agnóstico en FastAPI permite alternar entre proveedores cloud (Google Gemini, OpenAI) y ejecución 100% local (Ollama / Llama 3.2 3B).
 4. **Visualización React en 2D:** Renderizado dinámico de nodos y aristas mediante HTML5 Canvas (`react-force-graph-2d`) con experiencia de usuario fluida y paneles laterales descriptivos.
-

2. Marco Académico y Planteamiento del Problema

2.1 Problema Identificado

El descubrimiento de combinaciones de ingredientes (maridaje o *flavor pairing*) tradicionalmente ha dependido de la intuición empírica o de enciclopedias culinarias estáticas. Aunque existen teorías científicas de maridaje de sabores (compartir compuestos aromáticos clave), no existen herramientas web abiertas e interactivas en español que combinen:

- Exploración visual intuitiva en forma de grafo dinámico.
- Explicaciones organolépticas personalizadas en lenguaje natural.
- Búsqueda en tiempo real de recetas aplicadas.

2.2 Objetivos del Proyecto

- **Objetivo General:** Desarrollar una aplicación web full-stack funcional y escalable que permita explorar redes de sabores e interacciones de ingredientes asistida por Inteligencia Artificial.
- **Objetivos Específicos:**
 1. Diseñar un esquema relacional optimizado en PostgreSQL para modelar grafos bidireccionales de afinidad, con consultas de vecinos resueltas mediante índices compuestos.
 2. Implementar un pipeline offline en Python para la generación, validación y sanitización de un dataset sintético de afinidades culinarias.
 3. Crear una API REST en FastAPI con arquitectura limpia y abstracción del proveedor de LLM.
 4. Desarrollar una interfaz de usuario interactiva en React utilizando la librería `react-force-graph-2d`.
 5. Integrar autenticación JWT para áreas personalizadas de usuarios (guardar combinaciones favoritas y recetas).

2.3 Defensa de Decisiones de Diseño y Arquitectura

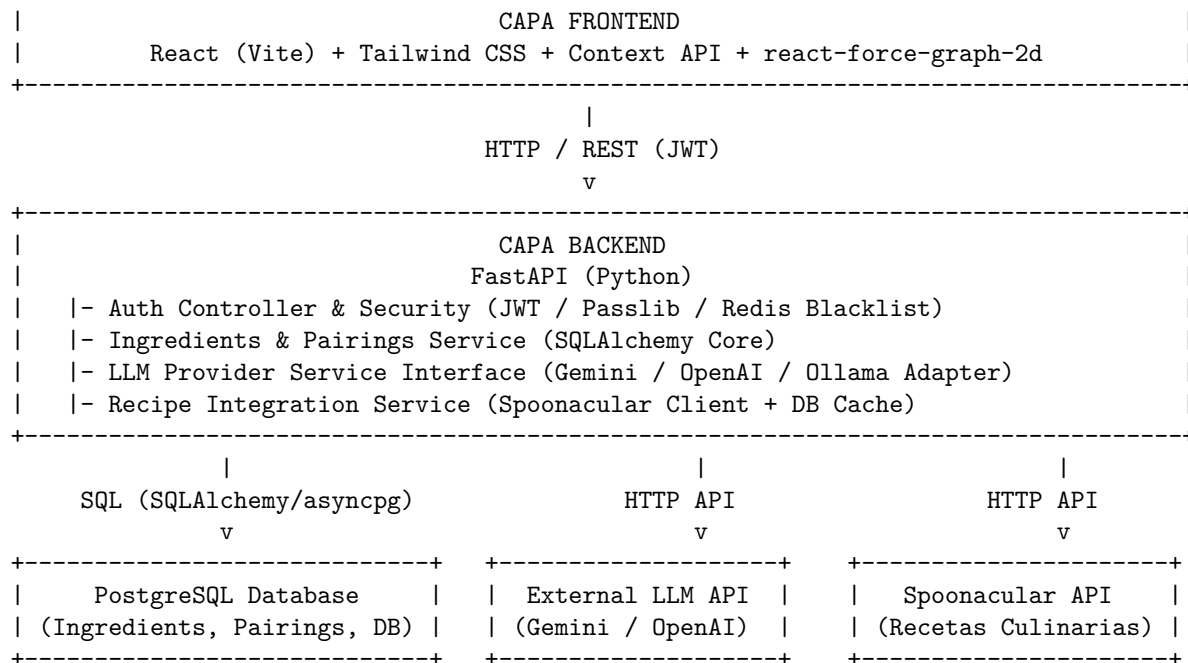
Las decisiones de diseño se fundamentan en criterios estratégicos de ingeniería de software:

- **PostgreSQL vs. Neo4j:** Se optó por PostgreSQL debido a que en una red de 300 a 1,000 ingredientes las consultas de 1 o 2 saltos (*hops*) no justifican la sobrecarga operativa y de consumo de memoria de un motor de grafos nativo como Neo4j. Mediante índices compuestos y ordenamiento de IDs (`ingredient_a_id < ingredient_b_id`), Postgres resuelve estas consultas de forma directa (sin recorridos recursivos) en **< 10ms**, con un costo operativo y de despliegue significativamente menor.
- **Arquitectura Híbrida de IA:** La IA no actúa como la base de datos (evitando alucinaciones o respuestas lentas en navegación UI), sino como un potenciador en dos fases: compilación de dataset en pipeline offline y generación de prosa culinaria en línea bajo demanda del usuario.

3. Arquitectura General del Sistema

El sistema utiliza un patrón de **Arquitectura Multicapa Desacoplada** (Frontend Client, Backend API, Relational Storage & External Services).

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+



3.1 Flujo de Datos Principal

1. **Carga Inicial del Grafo:** El cliente React solicita `GET /api/v1/graph`. FastAPI consulta PostgreSQL y retorna los nodos y enlaces activos (sub-grafo paginado).
2. **Exploración y Filtrado:** El usuario selecciona un nodo (ej: *Tomate*). El frontend resalta vecinos y solicita `GET /api/v1/ingredients/{id}/pairings`.
3. **Explicación con IA (Online):** Al presionar “¿Por qué combinan?”, el frontend invoca `POST /api/v1/ai/explain-pairing`. FastAPI utiliza la interfaz `LLMProvider` para generar un párrafo descriptivo con tono gastronómico.
4. **Recetas Relacionadas:** Al solicitar recetas para una combinación (ej: *Tomate + Albahaca + Ajo*), FastAPI consulta la caché local. Si no existe en caché, llama a la API de Spoonacular y guarda el resultado traducido.

4. Dataset de Sabores y Pipeline Offline

4.1 Origen y Fuentes de la Información

El dataset no se construye por relevamiento manual, sino combinando tres fuentes complementarias:

1. **La IA como Sintetizador y Destilador de Conocimiento (Método Principal):**

- Los modelos de lenguaje modernos (OpenAI GPT-4o, Google Gemini) fueron entrenados con millones de textos científicos, recetas globales y literatura gastronómica de referencia (incluyendo *The Flavor Bible*, *The Flavor Thesaurus* y artículos científicos de maridaje molecular).
- En lugar de descargar o transcribir libros, nuestro script en Python (`scripts/seed_flavor_network.py`) consulta en lote al LLM mediante solicitudes estructuradas (JSON Mode con Pydantic). La IA actúa como un “chef experto”, evaluando cada par de ingredientes y generando la puntuación de afinidad (0.0 a 1.0) y la explicación culinaria.

2. Datasets Abiertos Académicos:

- **FlavorDB / Flavornet:** Proyecto científico abierto de IIIT Delhi que mapea ~1,000 ingredientes a sus moléculas aromáticas volátiles (eugenol, linalool, etc.). Sus datasets son descargables públicamente en formato CSV/JSON.
- **Nature Scientific Reports - Dataset de “Flavor Network”:** Dataset público del famoso estudio científico de Yong-Yeol Ahn (“*Flavor network and the principles of food pairing*”).

3. Co-ocurrencia Estadística en Recetas (Spoonacular / RecipeDB):

- Mapeo estadístico automático: Si dos ingredientes (ej: *Tomate* y *Albahaca*) aparecen juntos frecuentemente en miles de recetas procesadas, se refuerza la puntuación de afinidad.

Nota metodológica sobre la normalización del score: Las fuentes anteriores no son directamente comparables entre sí. FlavorDB/Flavornet expresan afinidad como cantidad de compuestos aromáticos volátiles compartidos (un número entero), mientras que el LLM devuelve directamente un puntaje 0.0-1.0 y la co-ocurrencia en recetas es una frecuencia relativa. El pipeline define una fórmula explícita de normalización para llevar todas las fuentes a la misma escala (0.0 a 1.0) antes de promediarlas o combinarlas.

4.2 Fases del Pipeline Offline y Control de Alucinaciones

1. **Semilla de Ingredientes:** Listado inicial normalizado en JSON/CSV con ~250 ingredientes clasificados por categorías (Frutas, Verduras, Carnes, Lácteos, Hierbas/Especias, Granos).
2. **Generación Automatizada con `--dry-run`:** Generación de pares lógicos con opción de simulación sin gasto de cuota API.
3. **Suite de Verificación y Cola de Arbitraje:** Verificación contra matriz de incompatibilidad prohibida (~50 pares antagónicos como *Pescado + Dulce de Leche*). Pares dudosos se derivan a la cola `pairing_review_queue` para arbitraje manual mediante el comando CLI `./scripts/curate.py`.

5. Especificaciones del Servicio Backend y Recetas

5.1 Evaluación Multi-Ingrediente y Resiliencia de IA

- **Cálculo Determinístico:** `POST /api/v1/pairings/evaluate` calcula únicamente a partir de la base de datos la matriz de afinidades cruzadas $N \times N$, el **Índice de Sinergia Global (0-100%)** y el ingrediente discordante (*clashing element*).
- **Sugerencia Generativa:** `POST /api/v1/ai/suggest-replacement` sugiere bajo demanda un ingrediente alternativo para corregir el choque de sabor.
- **Jerarquía de Fallback (Degradación Grácil):** Llamadas a IA configuradas con **timeout estricto de 3.0s** y máximo 1 reintento. Si los proveedores cloud (Gemini/OpenAI) fallan, la API responde automáticamente con la justificación inmutable de PostgreSQL (`ai_rationale`), garantizando 100% de disponibilidad.

5.2 Servicio de Recetas y Traducción al Cachear (Translation-on-Cache)

- **Almacenamiento Permanente en Servidor:** Las recetas recuperadas de Spoonacular quedan guardadas en PostgreSQL para proteger la cuota gratuita (150 puntos/día) y reducir la latencia de 2000ms a < 10 ms.
- **Traducción Automática al Español:** Al descubrir una receta en inglés, el LLM traduce título, ingredientes e instrucciones **una sola vez antes de guardar en PostgreSQL**. Todas las consultas posteriores leen el texto traducido directamente desde la base de datos local.
- **Sanitización de Payload (30 KB max):** Middleware que remueve metadatos publicitarios y HTML innecesario de Spoonacular.

(Ver esquemas DDL completos, contratos Pydantic v2 y código de sanitización en *SPEC.md*)

6. Diseño Frontend y Experiencia Visual

El frontend se estructura con **React (Vite)** y **Tailwind CSS**.

Navbar: Logo Buscador: [Tomate x] [Albahaca x] Filtros [Evaluar]			
AREA PRINCIPAL DEL GRAFO (react-force-graph-2d)		Panel Lateral (Drawer)	
(QUESO)		Sinergia Global: 92%	
(Verde 92%)		[=====]	
		Matriz de Compatibilidad:	

	v		* Tomate + Albahaca: 98%	
	(TOMATE) ===== (ALBAHACA)		* Tomate + Queso: 92%	
	(Verde 98%)			
	: (Rojo punteado 35%)		[Explicación de Chef]	
	v		[14 Recetas]	
	(CHOCOLATE)			
+-----+				

6.1 Landing Page y Búsqueda Inicial

- **Buscador Central:** Pantalla limpia con sugerencias y etiquetas de tendencias ([Tomate y Albahaca], [Palta y Limón], [Chocolate y Naranja]).
- Transición fluida hacia la vista de exploración de grafo al seleccionar o tipear ingredientes.

6.2 Lógica Dinámica y Progresiva del Grafo (1, 2 y N Ingredientes)

1. **Selección de 1 Ingrediente (ej: *Tomate*):**
 - El nodo seleccionado se ubica en el centro.
 - Se despliegan aristas radiales hacia sus **Top 5-8 vecinos de mayor afinidad** (*Albahaca, Ajo, Mozzarella, Orégano, Aceite de Oliva*).
2. **Selección de 2 Ingredientes (ej: *Tomate + Albahaca*):**
 - Arista principal entre ambos coloreada por su afinidad (Verde para afinidad alta > 75%).
 - **Resaltado por Intensidad Armónica:** Los ingredientes vecinos conectados que presentan alta afinidad **con AMBOS ingredientes a la vez** se iluminan con **mayor intensidad visual** (brillo/opacidad 100%).
3. **Incorporación de Ingrediente Incompatible (ej: *Tomate + Albahaca + Chocolate*):**
 - La arista que conecta el ingrediente discordante (*Chocolate*) se grafica en **ROJO punteado** (incompatibilidad < 45%).
 - Los ingredientes vecinos alrededor del grupo se atenúan con **menor intensidad visual** (opacidad reducida al 30%).
4. **Escala Progresiva a N Ingredientes:** Recálculo en tiempo real (< 16ms / 60 FPS) al sumar o restar componentes a la receta.

6.3 Panel Lateral (Drawer) e Interacciones

- **Medidor Visual de Sinergia (Synergy Gauge):** Indicador porcentual del maridaje global del plato.
- **Matriz Interactiva $N \times N$:** Tabla de afinidades cruzadas para inspección rápida de pares.
- **Detector de Elemento Discordante (*Clashing Alert*):** Alerta en rojo identificando el ingrediente desentonante y habilitando sugerencia de reemplazo con IA.

7. Niveles de Suscripción y Modelo Freemium

Para garantizar la viabilidad comercial y el control de recursos del servidor, el sistema implementa dos niveles de cuenta:

Característica / Funcionalidad	Tier Gratuito (Free)	Tier Pago (Pro / Premium)
Límite de Ingredientes por Búsqueda Visualización de Grafo y Sinergia Explicación de Chef con IA	Hasta 3 ingredientes (ideal para tríadas gastronómicas)	Hasta 10 ingredientes (platos complejos / recetas completas)
Búsqueda de Recetas Guardado en Servidor (Workspace)	Incluido Acceso Completo	Incluido Acceso Completo
Exportación de Datos (Export & API)	Incluido (Límite diario)	Ilimitado
	Incluido Solo Combinaciones Favoritas	Incluido Combinaciones Favoritas + Recetas con Notas
	No disponible	Exportar a Texto Plano, JSON y Acceso a API Key

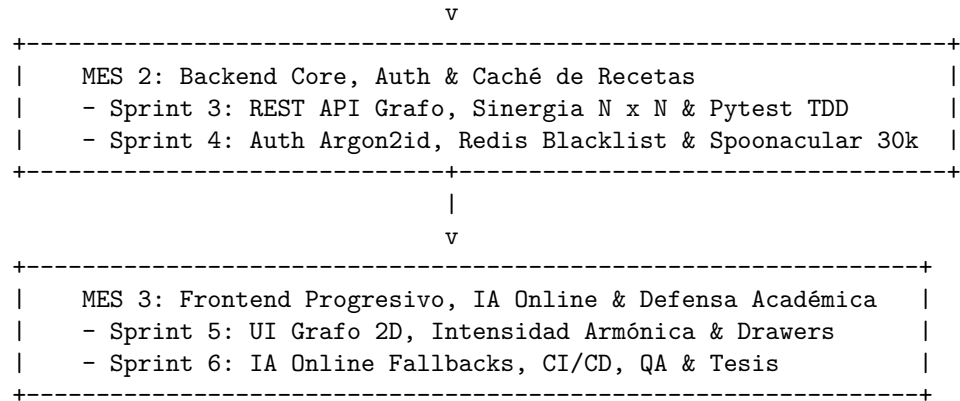
Justificación del Límite de 3 Ingredientes en el Tier Gratuito

Permite a los usuarios experimentar tríadas culinarias icónicas (ej: *Mirepoix*, *Tríada Caprese: Tomate + Albahaca + Mozzarella*), comprobando el valor de la sinergia. Para chefs profesionales o mixólogos que diseñan recetas complejas de 5 a 8 componentes, el **Tier Pro** desbloquea la capacidad total y la exportación de datos.

8. Plan de Implementación Ágil (Resumen de 3 Meses)

El desarrollo del proyecto se estructura en **3 Meses (12 Semanas)** divididos en **6 Sprints ágiles de 2 Semanas** (ver mapa detallado y *Definition of Done* por Sprint en PLAN.md):

+	-----	+
	MES 1: Cimientos, Pipeline de Datos & Prototipado	
	- Sprint 1: DDL Postgres, Redis, Alembic & Entorno Docker	
	- Sprint 2: Seed LLM (--dry-run) & Spike Grafo React 2D	
+	-----	+



9. Matriz de Riesgos y Análisis de Viabilidad Económica

9.1 Matriz de Riesgos

Riesgo Identificado	Impacto	Mitigación Planificada
Agotamiento de cuota en API de Recetas	Medio	Tabla caché en PostgreSQL (<code>recipe_search_cache</code>); depuración a 30 KB.
Saturación visual en el grafo	Alto	Subgrafo paginado (<code>GET /graph?limit=50</code>) y <code>min_affinity</code> por defecto en 0.50.
Alucinaciones o latencia en respuestas de IA	Medio	Prompting estructurado, suite <code>verify_coherence.py</code> , cola de revisión <code>pairing_review_queue</code> y fallback a <code>ai_rationale</code> .
Demoras en el objetivo opcional (Modelo Local)	Bajo	El modelo local es un <i>Stretch Goal</i> opcional; la arquitectura cloud se mantiene 100% funcional.

10. Análisis de Viabilidad Económica y Escalabilidad

El diseño arquitectónico del proyecto garantiza una **alta eficiencia de costos** y **sostenibilidad financiera**, permitiendo operar a costo **\$0.00 USD** durante

toda la fase de desarrollo y defensa, manteniendo costos marginales mínimos ante un crecimiento de usuarios a escala.

10.1 Estrategia de Optimización de Costos de API

1. **API de Recetas (Spoonacular) — Amortización por Caché Local:**
 - La cuota gratuita ofrece 150 puntos/día.
 - Gracias al **Caché Progresivo en PostgreSQL (recipe_search_cache)**, la dependencia de la API disminuye asintóticamente con el uso: más del 90% de las búsquedas frecuentes de usuarios leen directamente de la base de datos local (< 10ms) sin consumir cuota externa.
2. **APIs de Inteligencia Artificial (LLM) — Modelos de Alta Eficiencia:**
 - Se utilizan modelos de última generación ultralivianos (Google Gemini 1.5 Flash / OpenAI GPT-4o-mini).
 - **Generación en tiempo real:** \$0.075 USD por millón de tokens en Gemini Flash (~\$0.00005 USD por explicación de chef).
 - **Pipeline Offline (Mes 1):** Compilación inicial del grafo de 1,500 afinidades por **~\$0.30 USD por única vez**.

10.2 Cuadro Comparativo de Proyección de Costos por Nivel de Escala

Nivel de Escala	Usuarios Activos / Mes	API Recetas (Con Caché DB)	API IA (Gemini Flash / GPT-4o-mini)	Hosting & Post-greSQL	Costo Total Estimado
MVP / Defensa Tesis	1 – 100	\$0.00 (Free Tier)	\$0.00 (Free Tier)	\$0.00 (Render / Supabase Free)	\$0.00 USD / mes
Producción Inicial	1,000	\$0.00 (Caché DB absorbe 95%)	~\$0.15 USD	\$0 – \$5.00 USD	~\$0.15 – \$5.00 USD / mes
Escala Media	25,000	~\$29.00 USD (Spoonacular Builder)	~\$2.50 USD	~\$10.00 USD (DB 5GB)	~\$41.50 USD / mes

Referencias a la Documentación Técnica Completa

Para profundizar en los aspectos específicos de implementación y código:

* **SPEC.md:** Especificación técnica detallada con el DDL completo, scripts de sanitización (`sanitize_recipe_payload`), suite anti-alucinaciones (`verify_coherence.py`), Redis token revocation, contratos Pydantic v2 y pipeline de CI/CD en GitHub Actions. * **PLAN.md:** Plan de implementación detallado sprint por sprint, secuencia de kickoff en 5 pasos y archivos de infraestructura (`.env.example`, `docker-compose.yml`, `scripts/backup.sh`). * **README.md:** Portada del repositorio y guía de arranque rápido para desarrollo local.